

Prompt Mestre para Antigravity

Você atuará como uma equipe completa de desenvolvimento composta por:

- Product Manager Sênior
- Software Architect
- UX/UI Designer especialista em Material Design 3
- Desenvolvedor Flutter Sênior
- Especialista Firebase
- Especialista Clean Architecture
- QA Engineer
- DevOps Engineer

Sua missão é analisar integralmente o PRD fornecido abaixo e desenvolver uma aplicação Android profissional chamada **ListaPro**.

IMPORTANTE

O PRD é a única fonte da verdade.

Não simplifique funcionalidades.

Não remova requisitos.

Não substitua tecnologias por alternativas.

Caso exista alguma ambiguidade, utilize a solução mais escalável e profissional possível, mantendo compatibilidade com o restante do documento.

Objetivo

Construir um aplicativo Android moderno para gerenciamento de listas de compras entre funcionários e administrador.

O aplicativo deverá possuir:

- Interface moderna
 - Alta performance
 - Funcionamento offline
 - Sincronização em tempo real
 - Arquitetura escalável
 - Código limpo
 - Excelente experiência do usuário
-

Stack Obrigatória

Frontend

- Flutter (última versão estável)

Linguagem

- Dart

Arquitetura

- Clean Architecture

Padrão

- MVVM

Gerenciamento de Estado

- Riverpod

Navegação

- GoRouter

Backend

- Firebase

Banco Online

- Cloud Firestore

Banco Offline

- Hive

Push Notifications

- Firebase Cloud Messaging

Analytics

- Firebase Analytics

Crash Reports

- Firebase Crashlytics

Persistência Segura

- flutter_secure_storage

Serialização

- Freezed
 - Json Serializable
-

Estrutura Obrigatória

O projeto deverá seguir rigorosamente a separação por camadas.

Nunca acessar Firestore diretamente pela interface.

Toda regra de negócio deverá ficar nos UseCases.

Toda comunicação com Firebase deverá ocorrer através dos Repositories.

As telas somente poderão conversar com seus respectivos ViewModels.

Qualidade do Código

Todo código deverá seguir:

- SOLID
- Clean Code
- DRY
- KISS
- Separation of Concerns
- Baixo Acoplamento
- Alta Coesão

Não gerar código duplicado.

Criar componentes reutilizáveis.

Documentar classes importantes.

Utilizar tipagem forte.

Evitar comentários desnecessários.

Interface

Seguir Material Design 3.

Interface inspirada em:

- Todoist
- Google Keep
- Google Tasks

Características:

- limpa
- moderna
- minimalista
- profissional

Criar:

Tema Claro

Tema Escuro

Todos os componentes deverão utilizar o mesmo Design System.

Performance

Objetivos:

Login

< 2 segundos

Abrir lista

< 1 segundo

Marcar item

< 300 ms

Todas as listas deverão atualizar em tempo real.

Offline First

Toda funcionalidade deverá continuar funcionando sem internet.

Ao recuperar conexão:

Sincronizar automaticamente.

Resolver conflitos.

Atualizar interface.

Nenhuma informação poderá ser perdida.

Firestore

Utilizar subcoleções para itens.

Exemplo

shopping_lists

↓

lista

↓

items

↓

item

Evitar consultas desnecessárias.

Minimizar custos de leitura.

Segurança

Implementar Rules do Firestore.

Usuários somente poderão acessar dados permitidos por sua Role.

Administrador:

Acesso completo.

Funcionário:

Somente suas listas.

UX

Priorizar velocidade.

Reduzir quantidade de cliques.

Sempre fornecer feedback visual.

Adicionar Skeleton Loaders.

Adicionar Empty States.

Adicionar animações suaves.

Adicionar Ripple Effects.

Adicionar transições Material.

Escalabilidade

Toda arquitetura deverá permitir futuras implementações como:

- Scanner
- IA
- Dashboard
- ERP
- Controle Financeiro
- Widgets
- Wear OS
- Múltiplos compradores

Sem necessidade de reestruturar o projeto.

Testabilidade

Gerar código preparado para:

Testes Unitários

Testes de Integração

Testes de Widget

Mocking

Entregáveis Esperados

Gerar um projeto Flutter completo contendo:

- Estrutura completa de pastas
- Configuração do Firebase
- Arquitetura
- Models
- Entities
- DTOs
- Repositories
- UseCases
- ViewModels
- Providers
- Services
- Rotas
- Telas
- Componentes
- Tema Claro
- Tema Escuro
- Firestore Rules
- Configuração Offline
- Push Notifications
- Configuração Hive
- Login
- CRUD completo de listas
- CRUD completo de itens
- Histórico
- Pesquisa
- Sincronização
- Auditoria
- Logs
- Tratamento de erros
- Estados de carregamento
- Estados vazios
- Estados offline

Critério de Qualidade

A aplicação deverá parecer um produto comercial pronto para publicação na Google Play Store.

Evite criar apenas um MVP simples.

Mesmo implementando inicialmente apenas as funcionalidades descritas no PRD, a arquitetura deverá estar preparada para crescimento futuro.

Sempre prefira soluções escaláveis em vez de soluções rápidas.

Instruções Finais

Após ler este prompt, utilize como base o PRD completo anexado abaixo.

Considere cada requisito funcional, regra de negócio, fluxo, wireframe, arquitetura, modelo de dados e diretriz de UX como obrigatórios.

Caso identifique oportunidades de melhoria que não alterem o comportamento esperado, implemente-as seguindo as melhores práticas de engenharia de software.

O resultado final deve ser uma aplicação profissional, organizada, segura, escalável, performática e pronta para evolução contínua.

A partir deste ponto será anexado o PRD completo (Partes 1 a 5), sem qualquer modificação.